Day 3 – Vector Mathematics: guided lab and unguided assignment

Repository: `cosine-vector-mathematics` (local clone in `training/week1/day3`).

Checklist

☑ Part 1 guided: Steps 0–5	Step 1 12/12 · Step 3 4/4 PASS · Step 4 notes · repo tests 16/16
☑ Task A: by hand, prediction first	`docs/TaskA_hand_working.md` (*photo of handwritten working still to attach*)
☑ Task B: `nearest()`, tests, prompt log, one real issue found	12/12 own tests, Step 3 still 4/4
☑ Task C: scenario table, predictions, result	Use case #1 KYC document remediation
☑ Task D: decision cases	Table below
☑ Task E: misleading high score and own explanation	Below (*personal work example to be added*)
☑ Stretch: identity, centroid, sensitivity	`taskE_and_stretch.py`
☑ All data synthetic, nothing sensitive, no keys, no `.env`	

Part 1 – guided lab

Step	Result	Evidence
1 By hand	12/12 (E1–E4: dot, cosine, distance), worked on paper before typing	`STEP1_HAND_WORKING.md`, `outputs/step1_by_hand_check.txt`
2 Functions	`dot`, `norm`, `cosine_similarity`, `euclidean_distance`; standard library, a comment per line, answer key never opened	`my_vector_math.py`
3 Check	**4/4 PASS**	`outputs/step3_check_my_code.txt`
4 Why cosine	Four experiments, one takeaway each	`STEP4_NOTES.md`, `outputs/step4_why_cosine.txt`
5 Tests	16/16 after one fix: `test_stub_reports_todo` assumed the file was still a stub and failed once Step 2 was done; it now patches in a stub to test the TODO path	`tests/test_vector_math.py`

Task A – by hand again

Prediction (made before calculating): 1 KYC–AML, 2 overdraft–chargeback, 3 complaint–fraud.

Pair	Dot product	Cosine similarity	Euclidean distance	My prediction (1 = most similar)
overdraft and chargeback	22	0.773	3.606	2
KYC and AML	24	0.802	3.464	1
complaint and fraud	10	0.358	6.000	3

My prediction was right: the order matched, and cosine and distance agree on it. What intuition missed is how narrow the win is: KYC–AML (0.802) barely beats overdraft–chargeback (0.773), because AML's heavy fraud-risk

score pulls it away from KYC even though both are compliance controls. Full working: `docs/TaskA_hand_working.md`.

Task B – extend the code with Claude Code

`nearest(query, candidates, metric="cosine")` returns candidate names from most to least similar; cosine sorts high to low, euclidean low to high. **Version 1 (as first produced) silently treated any metric other than exactly "cosine" as euclidean**, so `"Cosine"` or the typo `"euclidian"` returned a confidently wrong ranking. Version 2 rejects unknown metrics, names the candidate when a comparison fails (for example a zero vector), and documents ties (input order is kept) and the empty case (`[]`).

Tests in `test_mine.py`: 5 for the four functions (Prompt 4), 7 for `nearest`. Each ranking test checks the full order, not just the length, so it can fail. Against v1, 5 failed. Four were real code problems, and **one was my own test's mistake**: I expected loan's list to end "fraud, complaint", but by hand complaint (0.370) beats fraud (0.344). On v2, 12/12 pass and `step3_check_my_code.py` still shows **4/4 PASS**. Full prompt log: `docs/TaskB_prompt_log.md`.

Task C – my scenario table

Use case **#1 KYC document remediation assistant**; dimensions identity / address / income proof and data-quality issue; six synthetic items (`data/my_scenario.json`).

	Predicted	**Result**
Most similar	Missing field ↔ Conflicting address	Cosine: Blurred ID scan ↔ Missing field **0.87** (my pick 0.85, 2nd). Distance: my pick is closest (3.00)
Least similar	Salary slip ↔ Conflicting address	Salary slip ↔ Blurred ID scan **0.12** (my pick 0.14)
Surprise	Utility bill ↔ Conflicting address	**0.69**, 3rd of 15 pairs: a valid document sits closer to a *problem* than to another valid document

The top two are 0.02 apart and the two metrics disagree on which is closest, so "the most similar pair" is not robust here. The surprise comes from my design: one vector mixes *what the document is* with *what is wrong with it*. I kept the numbers because they reflect my judgement, and noted that a real system should separate the two.

What the closest pair means to a bank team, and what a human still checks. Blurred ID scan and Missing field being closest tells the team the assistant sees both as "needs remediation", so they can share a queue and a playbook. A human still decides what is actually wrong and whether the customer is who they claim to be. A blurred ID can be an identity risk where a missing field is clerical, and no similarity score verifies an identity. Detail: `docs/TaskC_scenario.md`.

Task D – decision cases

Case	Metric and why	How I would choose the threshold	Who reviews / what is automated?
1. Knowledge-search assistant answers an agent's policy question, or says "I don't know"	**Cosine.** Text embeddings; policy clauses differ in length, and cosine compares topic, not size. On unit-length embeddings cosine and distance give the same ranking.	Build 100–200 real agent questions labelled with the correct clause, including some that have no answer in the policy set. For each candidate threshold, measure how often the right clause is returned and how often a wrong one is (above the line); pick the point where wrong answers are rare, accepting more "I don't know". Re-check monthly and whenever the policy library changes.	**Automated:** retrieve and answer *with the cited clause shown*. Below the threshold: "I don't know, ask a policy specialist". **Grey zone** (just above): show the clause but flag it "please confirm". A policy specialist reviews a weekly sample of answers and every agent-reported error. Because opposite clauses ("eligible" / "not eligible") can score close, combine it with keyword search and always show the clause text.
2. KYC tool compares an extracted address with the address on file and flags mismatches	**Not similarity alone.** Normalise both addresses first (case, abbreviations such as "Rd"/"Road", postcode format), then **exact match** on postcode and house number; use a **string-distance** score (character-level) for the rest. Embedding cosine is the wrong tool: "12 High Street" and "21 High Street" are near-identical in meaning but different homes.	Take a few hundred labelled pairs (same / different address) from past remediation cases. Set two lines: above the upper one = "match", below the lower one = "mismatch", in between = "review". Choose the upper line so that a missed true mismatch is extremely rare, because identity risk counts more than review effort.	**Automated:** only clear matches pass with no action. **Every** mismatch or grey-zone case goes to a KYC analyst, who checks the source document. The tool never auto-accepts an identity or address on a score, and every decision is logged with the score and the source.
3. Data pipeline suggests which two customer records may be duplicates	**Both, as features.** Cosine on embeddings of name + address text to find candidates quickly, then exact or rule checks on strong identifiers (date of birth, national ID, account number) and edit distance on names.	Label a few hundred candidate pairs from a past de-duplication exercise. Use precision at the top of the ranked list to set a *suggestion* threshold; a high-confidence band also needs all strong identifiers to agree. Measure the false-merge rate separately: a wrong merge mixes two customers' data and is far worse than a missed duplicate.	**Propose, never auto-merge.** Pairs above the threshold go to a data steward queue with the scores and the fields that agree or disagree. A merge needs steward sign-off, and a second approver for accounts with money or regulatory flags. Merges are reversible and logged.

Task E – where it breaks, and my own explanation

A misleading high score (`outputs/taskE_and_stretch.txt`, computed with my functions):

Space	Item A	Item B	Cosine
My KYC scenario	"Address matches records" [0, 5, 0, 2]	"Address does NOT match records" [0, 5, 0, 3]	**0.987**
Toy lending space	"Approve the loan" [5, 1, 2, 2]	"Decline the loan" [5, 2, 2, 2]	**0.987**

Opposite outcomes, a cosine of 0.987, and a distance of 1.

Why this happens with real embeddings, and the safeguard. Embeddings capture what a text is *about*: a loan decision, an address check. Two texts on the same subject share almost all of that, so they point the same way. The word that flips the meaning ("not", "decline", "reject") is a small part of the text and moves the vector only a little, so negations, opposite decisions and swapped numbers often score as near-duplicates. A similarity score therefore says "same topic", never "same decision". In a banking process I would never let a similarity score *decide* an outcome. A second, exact check must confirm the decision fields: a rule or regex for negation and outcome

words, an exact comparison of key fields (amount, status, address lines), or a cross-check model. Disagreements go to a human reviewer, and the source text is always shown next to the match.

In my own words: why closer vectors are treated as similar in meaning. An embedding model is trained so that texts used in similar ways, about the same things, end up with similar lists of numbers. Each number is a little like the dials in my toy vectors: how much "lending", how much "fraud". The direction the arrow points summarises the mix of topics. Two texts whose arrows point the same way have the same mix, so we treat them as meaning roughly the same thing. Cosine measures that angle and ignores how long the arrow is, which is mostly how long or wordy the text was. *Example from my own work (to personalise):* in a complaints queue, "my card payment was taken twice" and "duplicate debit on my account" share almost no words, but they describe the same problem, so their vectors point the same way and a search for one finds the other. "Payment was **not** taken twice" would land right next to them too, which is exactly why a person, or an exact rule, still has to read the result.

Stretch tasks

- **Unit vectors:** Blurred ID scan vs Missing field → cosine 0.8746; distance² of the unit vectors 0.2509 = 2 − 2 × 0.8746. The identity holds.
- **Centroid (drift idea):** added `centroid()`. Last month's centre (clean documents) [2.33, 2.33, 1.67, 0.00] versus this month's (problem items) [2.00, 2.00, 0.33, 4.33]: cosine 0.516, distance 4.56. Data quality went from 0 to 4.33, which is the kind of shift a drift alert should catch.
- **Sensitivity:** changing Utility bill by ±1 or 2 per dimension against Conflicting address (base 0.693). The biggest move is data quality (+0.21 for +2), the dimension where the *other* item is large. Raising a dimension the other item scores 0 on only lowers cosine. So a dimension's weight is not fixed: it depends on the other vector.